TP10

Objectif :

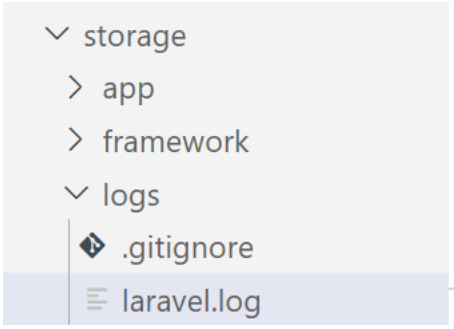
- Manipuler les middlewares
- Utiliser les sessions en Laravel

Exercice 1 :

1. Ajouter un middleware **JournalIpClient** permettant d'enregistrer les adresses ip des clients qui ont consulté notre application dans le journal (log) : Laravel.log.
2. Ajouter une route affichant une page « home » et utilisant le middleware JournalIpClient.

Indications :

La méthode **Log::info(« message »)** de la classe **Illuminate\Support\Facades\Log** permet d'ajouter une ligne dans le journal (log) situé sous le répertoire : storage/logs/laravel.log



Exercice 2 :

Ecrire un programme qui permet de gérer le temps d'inactivité d'un utilisateur connecté. Au cas où celui-ci dépassera une certaine durée, le programme le déconnectera.

1. Pour simplifier, on suppose que l'accès à l'application ne se fait qu'avec un seul compte : (username=« admin », password=« admin »)
2. Ajouter deux pages (en créant leur vue, contrôleur et route)
   a. **login** : permettant de vérifier les coordonnées de l'utilisateur et de remplir la session en cas de réussite ;
   b. **dashbord** : page affichant un simple message,
3. En utilisant un middleware « **dashbordAccessMiddleware** », ajouter une condition qui permet de vérifier si l'utilisateur est connecté sinon on le redirige vers la page « login ». Affecter ce middleware à la route « dashbord ».
4. On désire ajouter un code permettant de gérer le temps d'inactivité d'un utilisateur et le déconnecter automatiquement s'il est inactif pendant une période de temps donnée

a. Ajouter un middleware appelé **« SessionTimeoutMiddleware »** permettant de mettre à jour la dernière activité enregistrée dans la session à chaque demande et d'effacer la session si l'utilisateur est inactif pendant plus de 5 minutes ;
b. Protéger la route « dashbord » en lui affectant le middleware « *SessionTimeoutMiddleware* » ;
c. Ajouter le code nécessaire pour que la durée d'inactivité soit variable et soit passée en paramètre du middleware.

Indications :

Voici une liste non exhaustive des méthodes utilisées par Laravel pour manipuler ses sessions :

Méthode	Explication	Exemple
put($key, $value)	Cette méthode permet de stocker une donnée dans la session en utilisant une clé et une valeur. Si la clé existe déjà, elle sera remplacée par la nouvelle valeur. On peut utiliser : **session([$key => $value]);**	session()->**put**('username', 'john');
get($key, $default = null)	Cette méthode permet de récupérer la valeur d'une donnée stockée dans la session en utilisant une clé. Si la clé n'existe pas, elle retourne la valeur par défaut (qui est null par défaut).	$username = session()->get('username', 'guest');

has($key) :	Cette méthode permet de vérifier si une donnée existe dans la session en utilisant une clé.	if (session()->has('name'))
		{
		// La session contient la clé 'name'
		}

forget($key)	Cette méthode permet de supprimer une donnée stockée dans la session en utilisant une clé.	session()->forget('username');
flush()	Cette méthode permet de supprimer toutes les données stockées dans la session	session()->flush();
flash($key, $value)	Cette méthode permet de stocker une donnée dans la session pour une seule requête. La donnée sera supprimée de la session après la première utilisation.	session()->flash('message', 'Merci pour votre commande !');

reflash()	Cette méthode permet de conserver les données flash pour une requête supplémentaire. Utile si vous souhaitez rediriger l'utilisateur vers une autre page et garder les messages flash.	session()->reflash();
keep($keys)	Cette méthode permet de conserver certaines données flash pour la requête suivante. Utile si vous avez plusieurs messages flash et que vous souhaitez en conserver certains.	session()->keep(['message', 'error']);